南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Advanced Programming

Lab 3 Common Commands in Linux, Makefile

于仕琪，王大兴，廖琪梅, 王薇

CC BY NC SA

topics

- 1. Commands in Linux
  - Commands: directory, file, etc.
  - Commonly used commands, pipelines
- 2. Makefile
  - file: makefile/Makefile
  - command: make, make clean
- 3. Execises
  - commands, makefile
  - branch, loop

BY NC SA

1. Commands in Linux

1.1 Linux directory and file commands:

Command	Meaning
pwd	**P**rint the name of current/**w**orking **d**irectory.
cd <directory name>	**C**hange the current **d**irectory.
ls	**L**ist of content of a directory.
mkdir <directory name>	**M**ake a new **dir**ectory under any directory.
rmdir <directory name>	**Rem**ove **dir**ectories without files.
cat <file name>	Display content of the file.
rm <file name>	**Rem**ove a file.
cp <source> <dest>	**Co**py a file or files to another
mv<source><dest>	**Mov**e a file or files to another directory
cat/tail/head, less/more, nano/vim, file, whereis, echo	

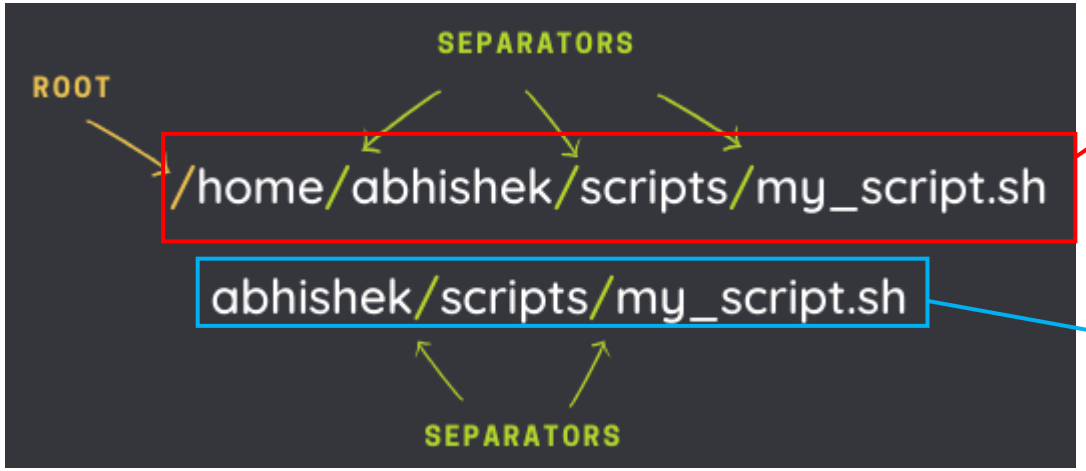
https://www.javatpoint.com/linux-commands

BY NC SA

Absolute path and relative path

A **path** is how you refer to files and directories. It gives the location of a file or directory in the Linux directory structure. It is composed of a **name** and **slash** syntax.

If the path starts with slash "/", the first slash denotes root. The rest of the slashes in the path are just separators.



The **absolute path** always starts from the root directory (/).

A **relative path** starts from the current directory.

Two special relative paths:

- **. (single dot) denotes the current directory in the path.**
- **.. (two dots) denotes the parent directory, i.e., one level above.**

BY NC SA

1.2 lists of commands, pipelines

- 1.2.1 lists of commands *https://www.gnu.org/software/bash/manual/bash.html#Lists*
  - An **AND list** has the form： command1 **&&** command2
    - ✓ **command2 is executed if, and only if, command1 returns an exit status of zero (success).**
  - An **OR list** has the form： command1 **||** command2
    - ✓ **command2 is executed if, and only if, command1 returns a non-zero exit status.**
  - **Command Sequence**: command1 **;** command2
    - ✓ **Commands separated by a ';' are executed sequentially;**
  - TIPS: *https://www.gnu.org/software/bash/manual/bash.html#Exit-Status*
    - ✓ For the shell's purposes, a command which **exits with a zero exit status has succeeded**. A non-zero exit status indicates failure.
    - ✓ The exit status of the last command is available in the special parameter **$?**

```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ ls
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ cp *.cpp ../
cp: cannot stat '*.cpp': No such file or directory
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ echo $?
1
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ cp *.cpp ../ && echo and_list
cp: cannot stat '*.cpp': No such file or directory
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ cp *.cpp ../ || echo or_list
cp: cannot stat '*.cpp': No such file or directory
or_list
```

CC BY NC SA

1.2 lists of commands, pipelines

- 1.2.2 pipelines https://www.gnu.org/software/bash/manual/bash.html#Pipelines
  - A pipeline is a sequence of one or more commands separated by one of the control operators '|' or '|&'.
  - The output of each command in the pipeline is connected via a pipe to the input of the next command. That is, each command reads the previous command's output. This connection is performed before any redirections specified by command1.

```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/src$ cat lab3_1.cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    char x=0xff;
    unsigned char y=0xff;
    cout<<hex<<x<<endl;
    cout<<hex<<y<<endl;
}ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/src$ cat lab3_1.cpp | grep cout
    cout<<hex<<x<<endl;
    cout<<hex<<y<<endl;
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/src$ cat lab3_1.cpp | grep include
#include <iostream>
#include <iomanip>
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/src$ cat lab3_1.cpp | grep include | gre
p iostream
#include <iostream>
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/src$
```

```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ ls ..
build  inc  lab3_1.c  lab3_2.c  src  test
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ echo .. | ls
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$ echo .. | xargs ls
build  inc  lab3_1.c  lab3_2.c  src  test
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab3/build$
```

CC BY NC SA

TIPS: Shortcut keys

- **Up** and **down** arrow keys can list the commands you typed.
- **Tab** key can complete the command. For a long command, you can type first few letters and press Tab key to complete the command or list alternate commands.

```
maydlee@LAPTOP-U1MOON2F:/mnt/d/CMake$ mkd
mkdir       mkdir.exe  mkdosfs
```

Type the first few letters of a command, and then press Tab key. If there is completion, press Tab key again, it will list the alternate commands.

clear is a standard Unix computer operating system command that is used to clear the terminal screen.

```
maydlee@LAPTOP-U1MOON2F:/mnt/d/CMake$ clear
```

BY NC SA

gcc & g++

gcc and **g++** are GNU C or C++ compilers respectively, which issued for preprocessing, compilation, assembly and linking of source code to generate an executable file.

Type command **gcc** or **g++ --help**, you can get the common options of the gcc or g++. **g++** accepts mostly the same options as **gcc**.

```
maydlee@LAPTOP-U1MOON2F: $ gcc --help
Usage: gcc [options] file...
Options:
  -pass-exit-codes         Exit with highest error code from a phase.
  --help                   Display this information.
  --target-help            Display target specific command line options.
  --help={common|optimizers|params|target|warnings|[^]{joined|separate|undocumented}}[,...].
                           Display specific types of command line options.
  (Use '-v --help' to display command line options of sub-processes).
  --version                Display compiler version information.
  -std=<standard>          Assume that the input sources are for <standard>.
  --sysroot=<directory>    Use <directory> as the root directory for headers
                           and libraries.
  -B <directory>           Add <directory> to the compiler's search paths.
  -v                       Display the programs invoked by the compiler.
  -###                     Like -v but options quoted and commands not executed.
  -E                       Preprocess only; do not compile, assemble or link.
  -S                       Compile only; do not assemble or link.
  -c                       Compile and assemble, but do not link.
  -o <file>                Place the output into <file>.
  -pie                     Create a dynamically linked position independent
                           executable.
  -shared                  Create a shared library.
```

CC BY NC SA

gcc & g++

-c Compile or assemble the source files, but do not link. The ultimate output is in the form of an object file for each source file. The object file name for a source file is made by replacing the suffix **.c** with **.o**.

-o *<file>* Place output in file *file*. This applies regardless to whatever sort of output is being produced, whether it be an executable file, an object file, an assembler file or preprocessed C code.
If **-o** is not specified, the default is to put an executable file in *a.out*.

gcc source_file.c -o program_name ***or*** ***gcc source_file.o -o program_name***

```
#include <stdio.h>
int main()
{
    printf("Hello World!\n");

    return 0;
}
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc -c hello.c
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
hello.c  hello.o  helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc -o hello hello.o
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
hello  hello.c  hello.o  helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./hello
Hello World!
```

compile

link

run

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ rm hello.o hello
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc hello.c -o hello
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
hello  hello.c  helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./hello
Hello World!
```

compile and link

run

CC BY NC SA

gcc & g++

With one step to generate an executable target file:

gcc file_name or ***g++ file_name***

This command is used to compile and create an executable file *a.out* (default target name).

```
#include <iostream>
using namespace std;
int main()
{
	cout << "Hello World!!!" << endl;

	return 0;
}
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc hello.c
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
a.out  hello  hello.c  helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./a.out
Hello World!
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ rm a.out hello
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ g++ helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
a.out  hello.c  helloworld.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./a.out
Hello World!!!
```

CC BY NC SA

gcc & g++

compile multiple files

You can compile the files one by one and then link them to an executable file.

Another choice is using one step to list all the .c(or .cpp) files after gcc(or g++) command and create an executable file named a.out.

```
//area.h
#define PI 3.1415

double compute_area(double r);
```

```
//area.c
#include "area.h"

double compute_area(double r)
{
   return PI * r * r;
}
```

```
//main.c
#include <stdio.h>
#include "area.h"

int main()
{
   double r, area;

   printf("Please input a radius:");
   scanf("%lf", &r);

   area = compute_area(r);

   printf("The area of %lf is %.4lf\n", r, area);

   return 0;
}
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc -c area.c
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc -c main.c
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
area.c  area.h  area.o  hello.c  helloworld.cpp  main.c  main.o
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc -o main main.o area.o
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ls
area.c  area.h  area.o  hello.c  helloworld.cpp  main  main.c  main.o
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./main
Please input a raduis:4.8
The area of 4.800000 is 72.3802
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ gcc area.c main.c
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/lab_example/lab03$ ./a.out
Please input a raduis:2.3
The area of 2.300000 is 16.62
```

CC BY NC SA

Makefile

What is a Makefile?

Makefile is a tool to simplify and organize compilation. **Makefile is a set of commands with variable names and targets .** You can compile your project(program) or only compile the update files in the project by using Makefile.

BY NC SA

Suppose we have four source files as follows:

multifiles > functions.h > ...

```cpp
#pragma once

#define N 5

void printinfo();
int factorial(int n);
```

multifiles > printinfo.cpp > ...

```cpp
#include <iostream>
#include "functions.h"

void printinfo()
{
    std::cout << "Let's go!" << std::endl;
}
```

multifiles > factorial.cpp > factorial(int)

```cpp
#include "functions.h"

int factorial(int n)
{
    if(n == 1)
        return 1;
    else
        return n * factorial(n-1);
}
```

multifiles > main.cpp > ...

```cpp
#include <iostream>
#include "functions.h"
using namespace std;

int main()
{
    printinfo();

    cout << "The factorial of "<< N << " is:" << factorial(N) << endl;

    return 0;
}
```

Normally, you can compile these files by the following command:

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/makefile/multifiles$ g++ -o testfiles main.cpp printinfo.cpp factorial.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/makefile/multifiles$ ./testfiles
Let's go!
The factorial of 5 is:120
```

or

```
g++ *.cpp
```

CC BY NC SA

How about if there are hundreds of files to compile? If only one source file is modified, need we compile all the files? Makefile will help you.

The name of makefile must be either **makefile** or **Makefile** without extension. You can write makefile in any text editor. A rule of makefile including three elements: **targets**, **prerequisites** and **commands**. There are many rules in the makefile.

BY NC SA

A makefile consists of a set of rules. A rule including three elements: **target**, **prerequisites** and **commands**.

```
targets : prerequisites
<TAB> command
```

- The **target** can be an object file, which will be generated by the command.
- The **prerequisites** are file names, separated by spaces. The targets generation depends on the prerequisites.
- The **commands** will be run to generate targets.

These need to start with a **tab**, not spaces.

BY NC SA

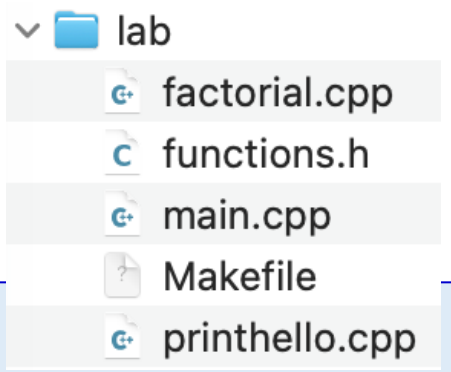
Makefile example

- lab
  - factorial.cpp
  - functions.h
  - main.cpp
  - Makefile
  - printhello.cpp

CC BY NC SA

Exercises 1. Makefile and make

```
/* functions.h */
#ifndef _FUNCTIONS_H_
#define _FUNCTIONS_H_
void printhello();
int factorial(int n);
#endif
```




```
/* main.cpp */
#include <iostream>
#include "functions.h"
using namespace std;
int main()
{
    printhello();
    cout << "This is main:" << endl;
    cout << "The factorial of 5 is: " << factorial(5) << endl;
    return 0;
}
```

Write a simple Makefile, which can:

- Generate object files.
- Generate an executable file.
- Remove all generated files by running "`make clean`".

```
/* factorial.cpp */
#include "functions.h"
int factorial(int n)
{
    if (n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```

```
/* printhello.cpp */
#include <iostream>
#include "functions.h"
using namespace std;
void printhello()
{
    int i = 0;
    cout << "Hello World!" << endl;
}
```

BY NC SA

Exercises

2. Run the following source code and explain the result.

```
#include <iostream>
using namespace std;

int main()
{
    for(size_t n = 2; n >= 0; n--)
        cout << "n = " << n << "  ";

    return 0;

}
```

BY NC SA

Exercises

3. Run the following source code and explain the result.

```
#include <iostream>
using namespace std;

int main()
{
    int n,fa;

    do{
        fa *= n;
        n++;
    }while(n <= 10);

    cout << "fa = " << fa << endl;

    return 0;
}
```

BY NC SA